



Transformers Explained

Sections

Machine Translation

- Seq2Seq Models with RNN
- Seq2Seq Models with Attention

Transformers

- A High-Level Look
- Encoders
- Self-Attention
- Multi-Headed Attention
- Positional Encoding
- Residuals & Layer Normalization
- Decoders
- Linear and Softmax Layer

1

Machine Translation

Seq2Seq Models with RNN



Seq2Seq Model

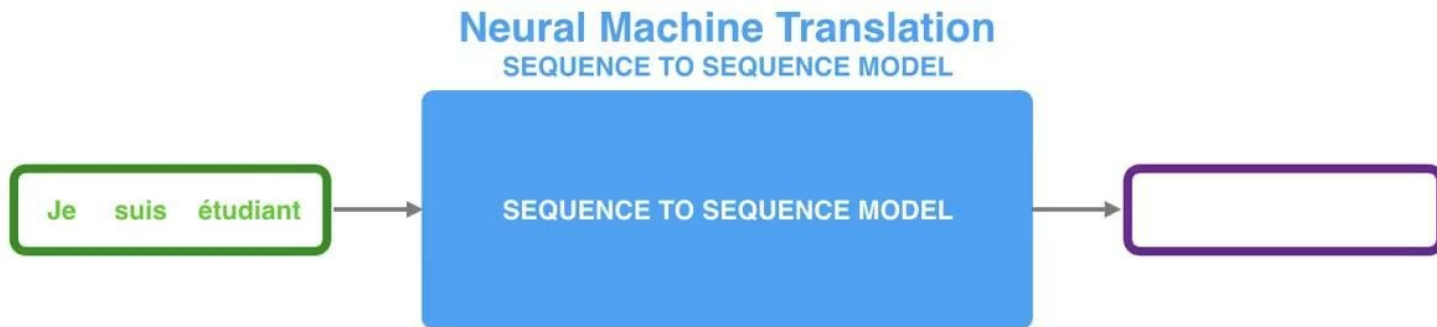
A sequence-to-sequence model is a model that takes a sequence of items (words, letters, features of an images...etc) and outputs another sequence of items.





Translation

In neural machine translation, a sequence is a series of words, processed one after another. The output is, likewise, a series of words.

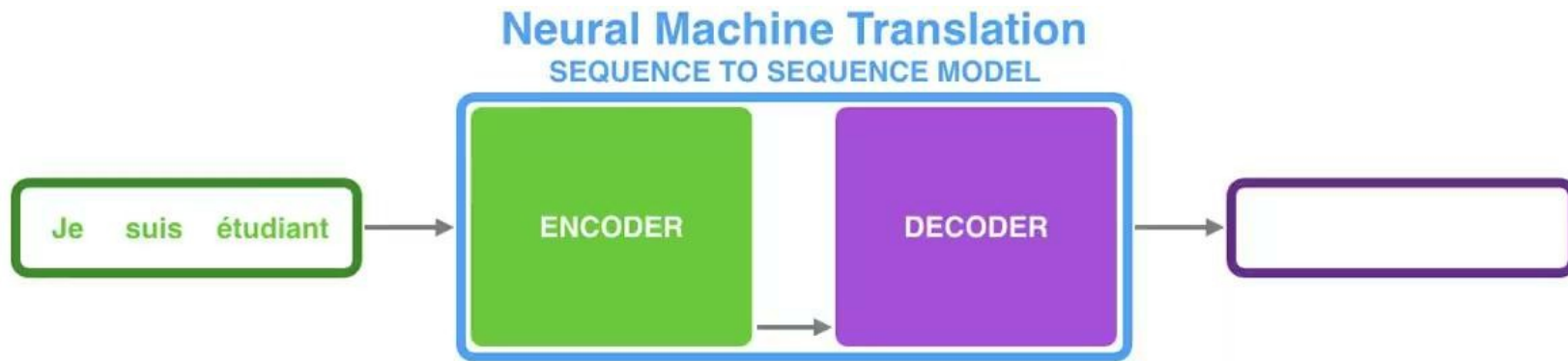




Under the hood

The encoder processes each item in the input sequence, it compiles the information it captures into a vector (called the **context**).

After processing the entire input sequence, the encoder sends the context over to the decoder, which begins producing the output sequence item by item.





Some Important Information

The Context

- The context is a vector (an array of numbers, basically) in the case of machine translation.
- You can set the size of the context vector when you set up your model. It is basically the number of hidden units in the encoder RNN.

The Input Sequence

- We need to turn the input words into vectors before processing them.
- That transformation is done using word embeddings.
- These turn words into vector spaces that capture a lot of the meaning/semantic information of the words
- We can use pre-trained embeddings or train our own embedding on our dataset.
- Embedding vectors of size 200 or 300 are common.



RNNs

By design, a RNN takes two inputs at each time step: an input (in the case of the encoder, one word from the input sentence), and a hidden state.

The next RNN step takes the second input vector and hidden state #1 to create the output of that time step, and so on.

Recurrent Neural Network

Time step #1:

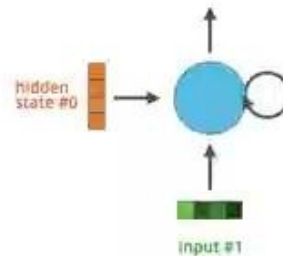
An RNN takes two input vectors:



hidden state #0



input vector #1



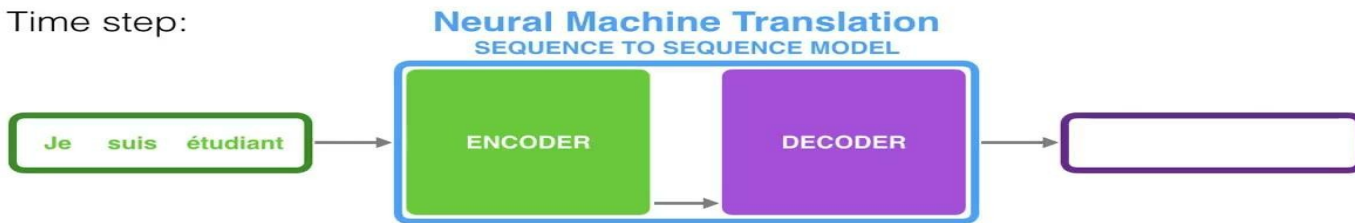


RNN in Seq2Seq

Now, let's see how that maps to an encoder-decoder architecture.

Notice how the **last hidden state** is actually the context we pass along to the decoder.

Time step:

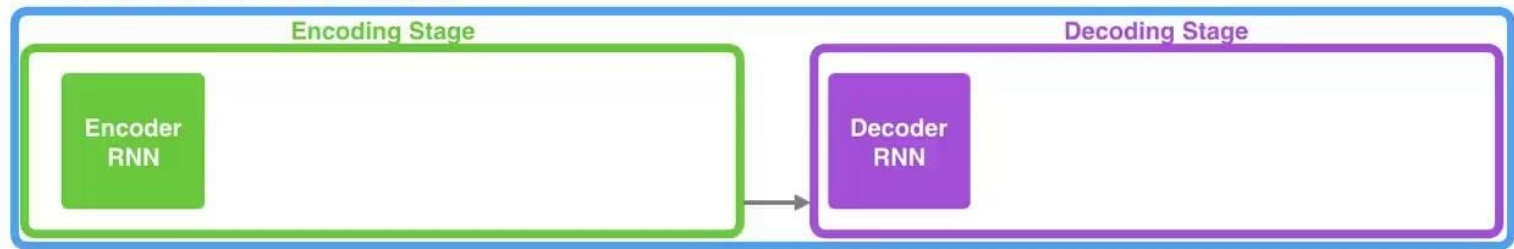




RNN in Seq2Seq

Unrolled View

Neural Machine Translation SEQUENCE TO SEQUENCE MODEL



Je

suis

étudiant



The Challenge

- The context vector turned out to be a bottleneck for these types of models.
- It made it challenging for the models to deal with long sentences.
- The fixed size of the context vector was an issue, as well.

2

Machine Translation

Seq2Seq Models with Attention



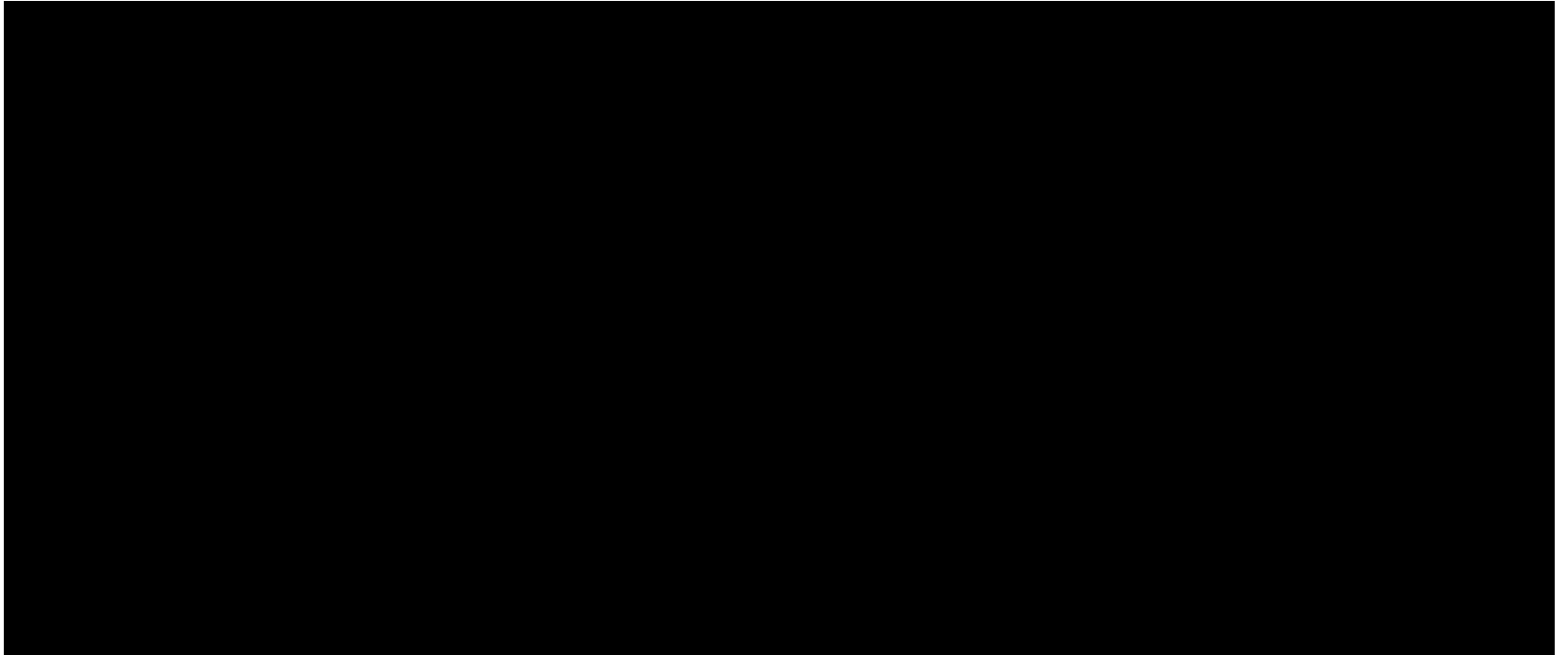
The Attention Mechanism

An seq2seq attention model differs from a classic sequence-to-sequence model in two main ways.

First, the encoder passes a lot more data to the decoder. Instead of passing the last hidden state of the encoding stage, the encoder passes all the hidden states to the decoder



The Attention Mechanism





The Attention Mechanism

Second, an attention decoder does an extra step before producing its output. In order to focus on the parts of the input that are relevant to this decoding time step, the decoder does the following:

1. Look at the set of encoder hidden states it received – each encoder hidden state is most associated with a certain word in the input sentence
2. Give each hidden state a score (let's ignore how the scoring is done for now)
3. Multiply each hidden state by its softmaxed score, thus amplifying hidden states with high scores, and drowning out hidden states with low scores



The Attention Mechanism

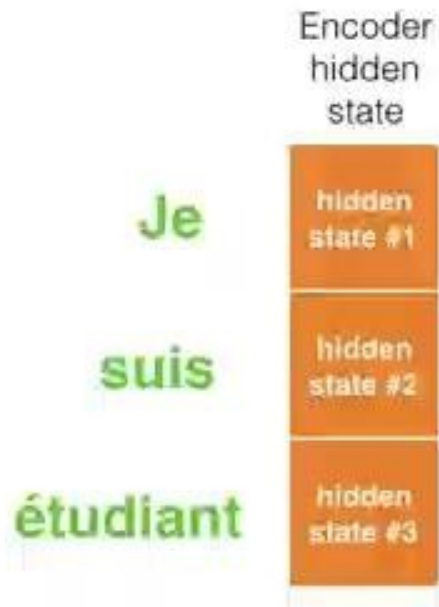
Attention at time step 4





The Attention Mechanism

This is another way to look at which part of the input sentence we're paying attention to at each decoding step:



3

Transformers

A High-Level Look



The Transformer Black Box

Let's begin by looking at the model as a single black box. In a machine translation application, it would take a sentence in one language, and output its translation in another.

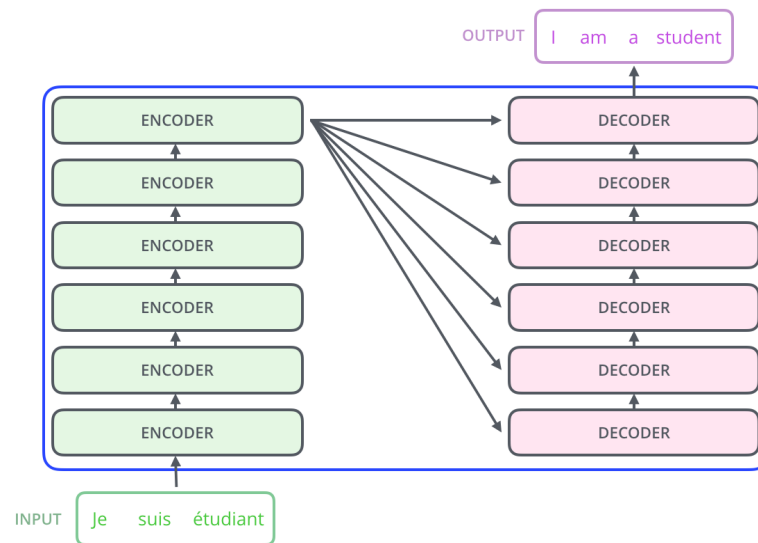




The Transformer Stack

The encoding component is a stack of encoders.

The decoding component is a stack of decoders of the same number.



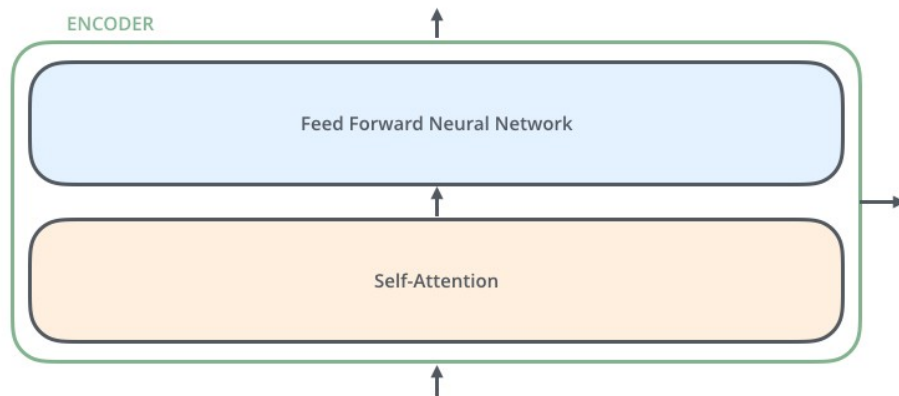


Encoder Structure

The encoders are all identical in structure. Each one is broken down into two subsections.

The encoder's inputs first flow through a self-attention layer – a layer that helps the encoder look at other words in the input sentence as it encodes a specific word.

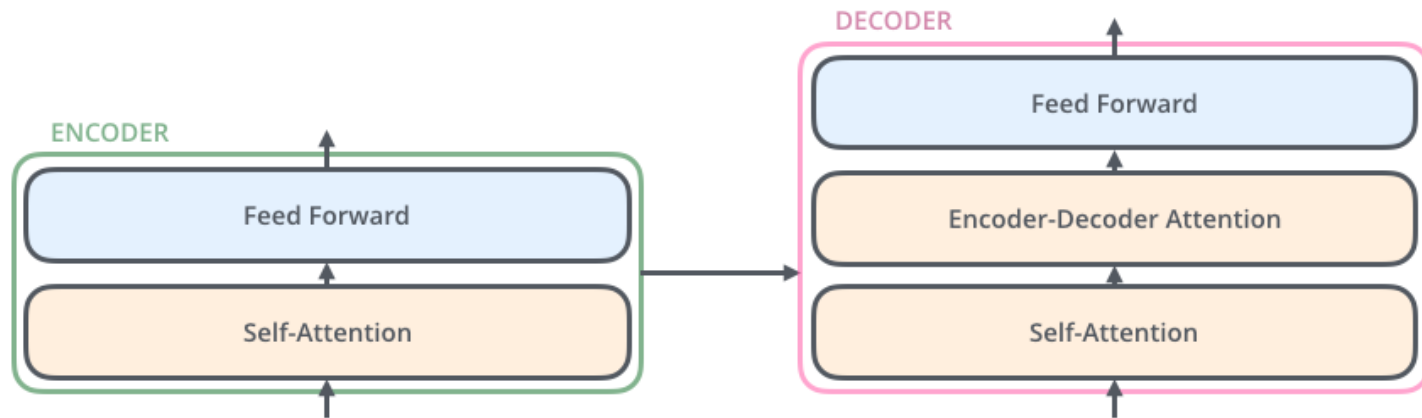
The outputs of the self-attention layer are fed to a feed-forward neural network.





Decoder Structure

The decoder has both those layers, but between them is an attention layer that helps the decoder focus on relevant parts of the input sentence (similar what attention does in seq2seq models).



4

Transformers

Encoders



Embeddings

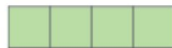
Now that we've seen the major components of the model, let's start to look at the various vectors/tensors and how they flow between these components to turn the input of a trained model into an output.

As is the case in NLP applications in general, we begin by turning each input word into a vector using an embedding algorithm.

Each word is embedded into a vector of size 512.

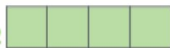
The embedding only happens in the bottom-most encoder.

x_1



Thinking

x_2



Machines



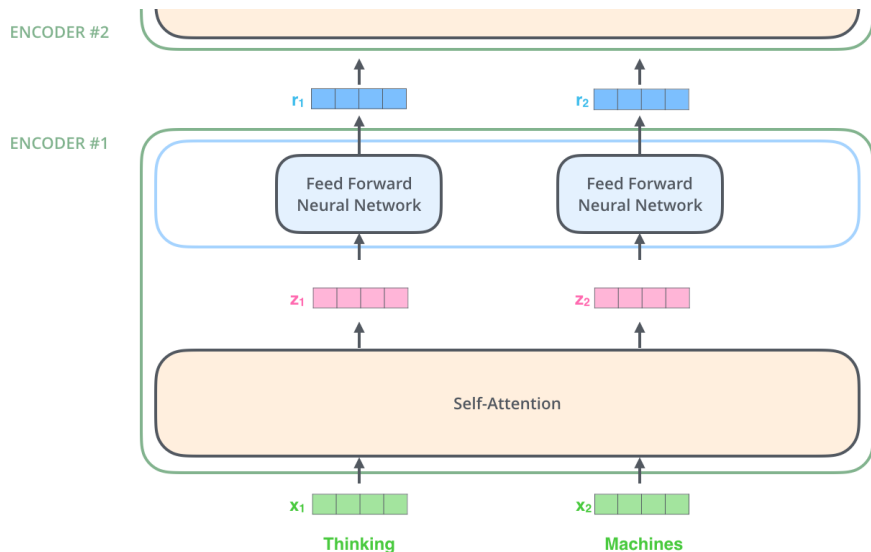
Encoders - Detail

After embedding the words in our input sequence, each of them flows through each of the two layers of the encoder.

Here we begin to see one key property of the Transformer, which is that **the word in each position flows through its own path in the encoder.**

There are dependencies between these paths in the self-attention layer.

The feed-forward layer does not have those dependencies, however, and thus the various paths can be executed in parallel while flowing through the feed-forward layer.





Encoders - Summary

An encoder receives a list of vectors as input. It processes this list by passing these vectors into a 'self-attention' layer, then into a feed-forward neural network.

Then it sends out the output upwards to the next encoder.



Before we talk about Self-Attention

During training, three matrices of weights are created (instead of just one).

- W_Q : Transforms the input into **queries** that capture what each token is "asking" about the other tokens.
- W_K : Transforms the input into **keys** that represent the "content" of each token that can be matched with queries.
- W_V : Transforms the input into **values** that carry the actual data to be used when producing the output.

These weight matrices are learned parameters of the model. They are typically initialized randomly at the beginning of training and then optimized using backpropagation during the training process.

5

Transformers

Self-Attention



Self-Attention

Say the following sentence is an input sentence we want to translate:

“The animal didn't cross the street because it was too tired”

What does “it” in this sentence refer to? Is it referring to the street or to the animal? It’s a simple question to a human, but not as simple to an algorithm.

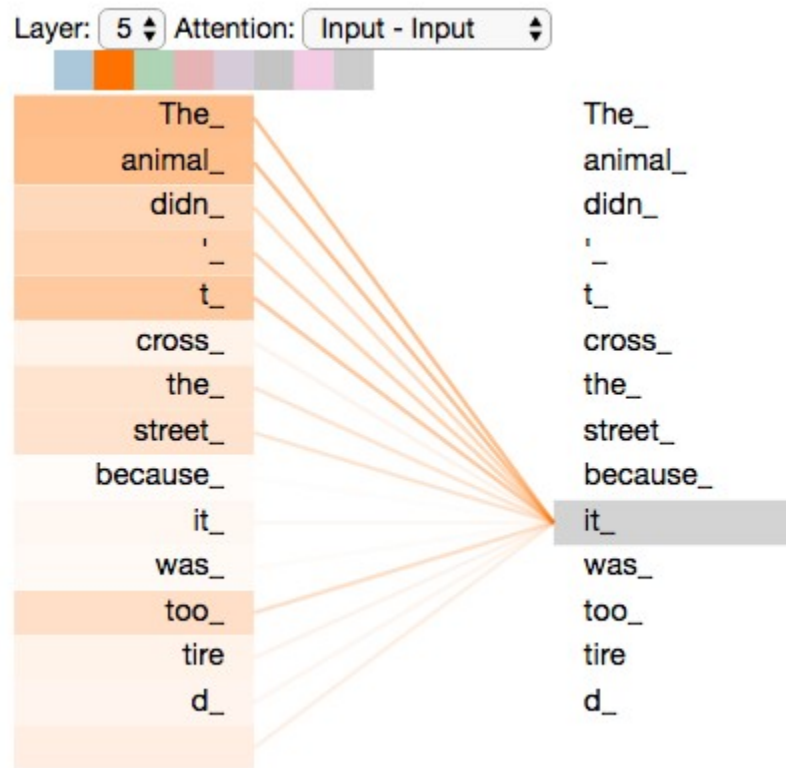
When the model is processing the word “it”, self-attention allows it to associate “it” with “animal”.

As the encoder model processes each word (each position in the input sequence), self attention allows it to look at other positions in the input sequence for clues that can help lead to a better encoding for this word.



Self-Attention

Self-attention is the method the Transformer uses to bake the “understanding” of other relevant words into the one we’re currently processing.



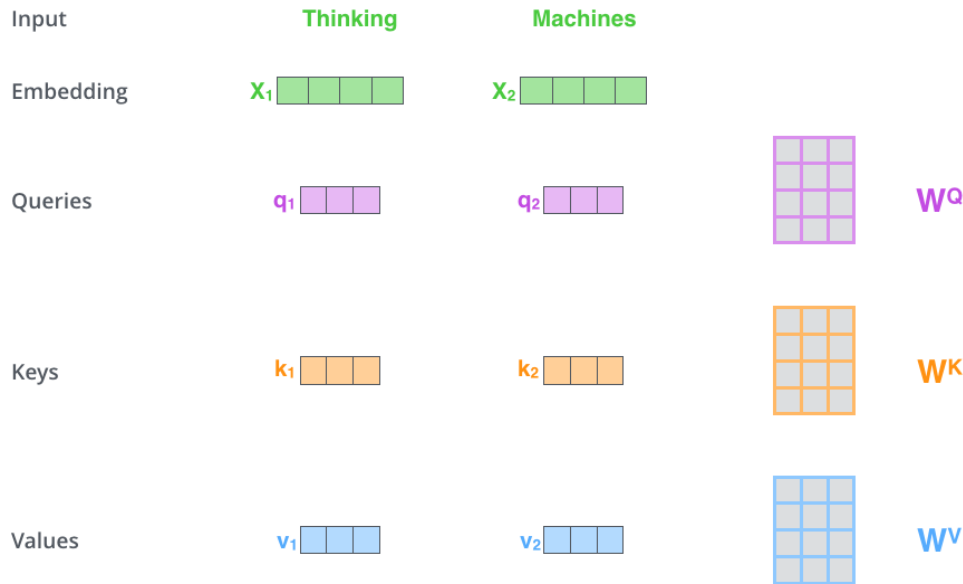


Self-Attention: Step 1

The first step in calculating self-attention is to create three vectors from each of the encoder's input vectors (in this case, the embedding of each word).

So for each word, we create a Query vector, a Key vector, and a Value vector.

These vectors are created by multiplying the embedding by three matrices that we trained during the training process.





Self-Attention: Step 2

The second step in calculating self-attention is to calculate a score. Say we're calculating the self-attention for the first word in this example, "Thinking".

We need to score each word of the input sentence against this word. The score determines how much focus to place on other parts of the input sentence as we encode a word at a certain position.

The score is calculated by taking the dot product of the query vector with the key vector of the respective word we're scoring.

So if we're processing the self-attention for the word in position #1, the first score would be the dot product of q_1 and k_1 . The second score would be the dot product of q_1 and k_2 .

Input

Embedding

Queries

Keys

Values

Score

Thinking

x_1

q_1

k_1

v_1

$q_1 \cdot k_1 = 112$

Machines

x_2

q_2

k_2

v_2

$q_1 \cdot k_2 = 96$



Self-Attention: Step 3,4

The third and fourth steps are to divide the scores by 8.

The square root of the dimension of the key vectors used in the paper – 64. This leads to having more stable gradients. There could be other possible values here, but this is the default.

Then pass the result through a softmax operation. Softmax normalizes the scores so they're all positive and add up to 1.

This softmax score determines how much each word will be expressed at this position. Clearly the word at this position will have the highest softmax score, but sometimes it's useful to attend to another word that is relevant to the current word.

Input

Embedding

Queries

Keys

Values

Score

Divide by 8 ($\sqrt{d_k}$)

Softmax

Thinking

x_1

q_1

k_1

v_1

$$q_1 \cdot k_1 = 112$$

14

0.88

Machines

x_2

q_2

k_2

v_2

$$q_1 \cdot k_2 = 96$$

12

0.12



Self-Attention: Step 5, 6

The fifth step is to multiply each value vector by the softmax score (in preparation to sum them up). The intuition here is to keep intact the values of the word(s) we want to focus on, and drown-out irrelevant words (by multiplying them by tiny numbers like 0.001, for example).

The sixth step is to sum up the weighted value vectors. This produces the output of the self-attention layer at this position (for the first word).

That concludes the self-attention calculation. The resulting vector is one we can send along to the feed-forward neural network.

Input

Embedding

Queries

Keys

Values

Score

Divide by $8 (\sqrt{d_k})$

Softmax

Softmax

X
Value

Sum

Thinking

x_1

q_1

k_1

v_1

$q_1 \cdot k_1 = 112$

14

0.88

v_1

z_1

Machines

x_2

q_2

k_2

v_2

$q_2 \cdot k_2 = 96$

12

0.12

v_2

z_2

6

Transformers

Multi-Headed Attention



What does head mean here?

Head = One individual, independent self-attention execution run.

Each head independently computes attention using its own set of weight matrices, allowing the model to capture different aspects or relationships within the input sequence.

By having multiple heads, the model can learn to focus on different aspects or relationships in the data. For instance, one head might focus on short-range dependencies (e.g., adjacent words in a sentence), while another head might focus on long-range dependencies (e.g., connections between words that are far apart). Yet another head could focus on specific keywords or entities, like names or dates, to understand specific relationships.

By combining the outputs of multiple heads, the model can capture a richer and more nuanced representation of the input data.

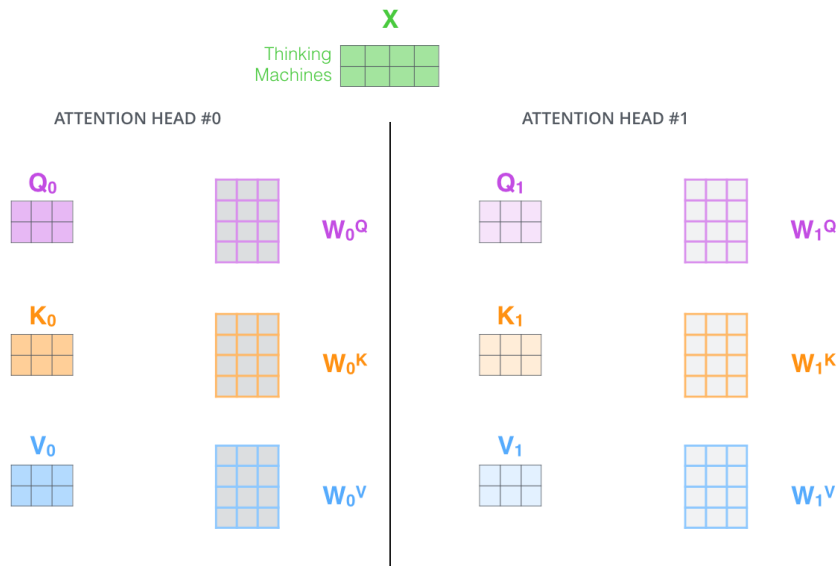
Each "head" is like a different lens through which the model views the input data, and by combining these views, the model gains a more comprehensive understanding.



Multi-Headed Attention

With multi-headed attention, we maintain separate Q/K/V weight matrices for each head resulting in different Q/K/V matrices. As we did before, we multiply X by the $W_Q/W_K/W_V$ matrices to produce Q/K/V matrices.

It expands the model's ability to focus on different positions. Yes, in the earlier example, z_1 contains a little bit of every other encoding, but it could be dominated by the actual word itself. If we're translating a sentence like "The animal didn't cross the street because it was too tired", it would be useful to know which word "it" refers to.



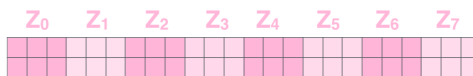


Multi-Headed Attention

This leaves us with a bit of a challenge. The feed-forward layer is not expecting eight matrices – it's expecting a single matrix (a vector for each word). So we need a way to condense these eight down into a single matrix.

How do we do that? We concat the matrices then multiply them by an additional weights matrix W_O .

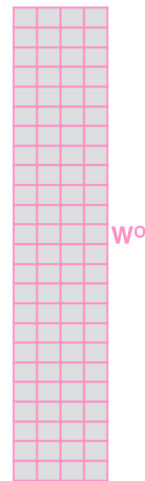
1) Concatenate all the attention heads



2) Multiply with a weight matrix W^O that was trained jointly with the model

X

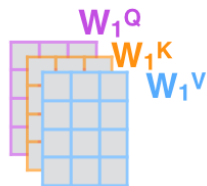
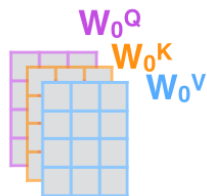
3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN



Multi-Headed Self-Attention

- 1) This is our input sentence*
- 2) We embed each word*
- 3) Split into 8 heads. We multiply X or R with weight matrices
- 4) Calculate attention using the resulting $Q/K/V$ matrices
- 5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer

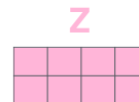
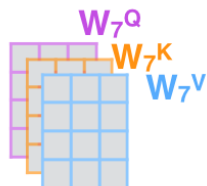
Thinking
Machines



...

...

...



* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one



7

Transformers

Positional Encoding



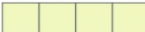
Positional Encodings

The transformer adds a vector to each input embedding.

These vectors follow a specific pattern that the model learns, which helps it determine the position of each word, or the distance between different words in the sequence.

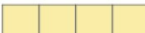
The intuition here is that adding these values to the embeddings provides meaningful distances between the embedding vectors once they're projected into Q/K/V vectors and during dot-product attention.

EMBEDDING
WITH TIME
SIGNAL

x_1 

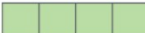
=

POSITIONAL
ENCODING

t_1 

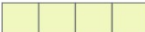
+

EMBEDDINGS

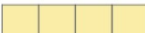
x_1 

INPUT

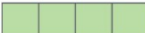
Je

x_2 

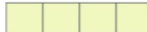
=

t_2 

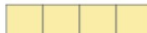
+

x_2 

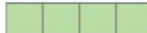
suis

x_3 

=

t_3 

+

x_3 

étudiant

8

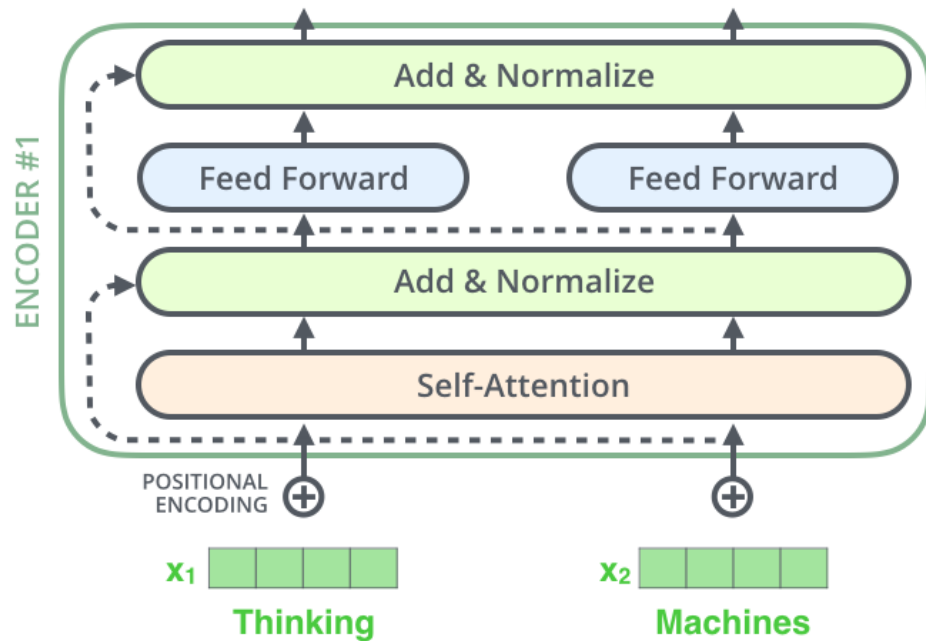
Transformers

Residuals & Layer Normalization



Residuals & Layer Norm

One detail in the architecture of the encoder that we need to mention before moving on, is that each sub-layer (self-attention, ffnn) in each encoder has a residual connection around it, and is followed by a layer-normalization step.





Residuals & Layer Norm

Residual Connections:

- A residual connection, also known as a skip connection, is a shortcut path that allows the input to bypass a layer and be added directly to the output of that layer.
- Purpose: The purpose of using residual connections is to help mitigate the vanishing gradient problem and to facilitate the flow of information and gradients during training. By allowing gradients to propagate more effectively through the network, residual connections enable deeper architectures to be trained more successfully.

Layer Normalization:

- Layer normalization is a technique used to stabilize and accelerate the training of deep neural networks. It normalizes the output of each layer to have a mean of zero and a standard deviation of one.
- Purpose: The purpose of layer normalization is to ensure that the outputs of the layers maintain a consistent scale and distribution, which helps in speeding up convergence and improving overall model performance.

9

Transformers

Decoders



Decoders

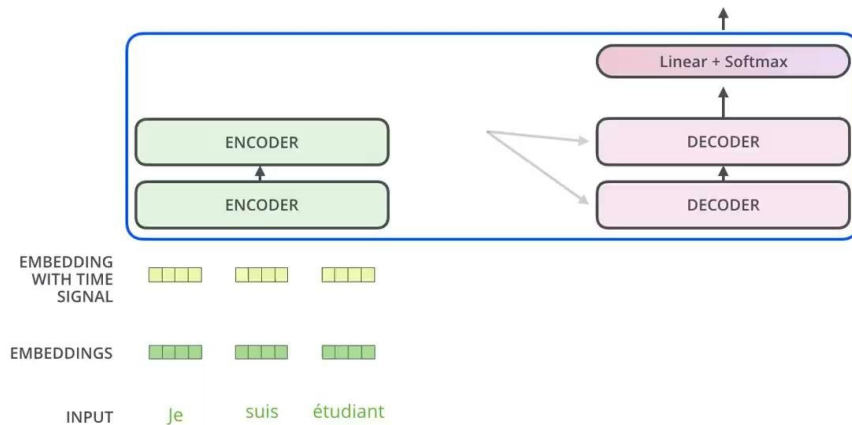
Now that we've covered most of the concepts on the encoder side, we basically know how the components of decoders work as well. But let's take a look at how they work together.

The encoder starts by processing the input sequence. The output of the top encoder is then transformed into a set of attention vectors K and V .

These are to be used by each decoder in its “encoder-decoder attention” layer which helps the decoder focus on appropriate places in the input sequence.

Decoding time step: 1 2 3 4 5 6

OUTPUT



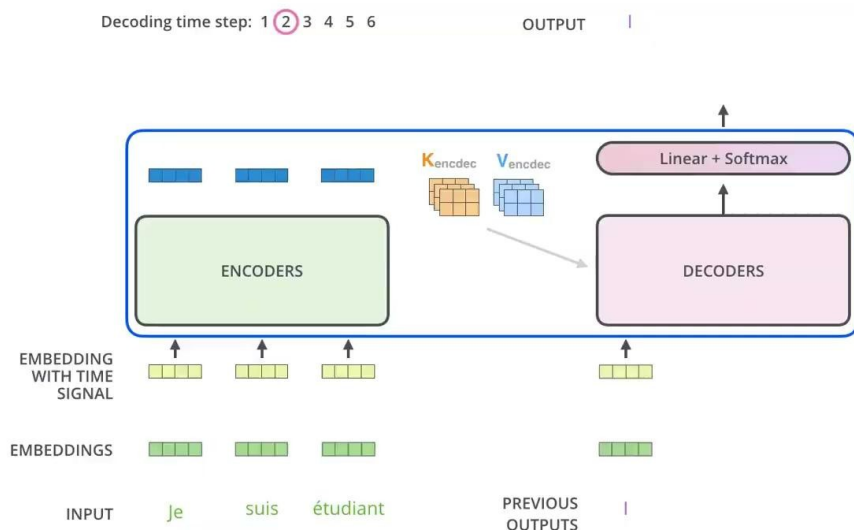


Decoders

The following steps repeat the process until a special symbol is reached indicating the transformer decoder has completed its output.

The output of each step is fed to the bottom decoder in the next time step, and the decoders bubble up their decoding results just like the encoders did.

And just like we did with the encoder inputs, we embed and add positional encoding to those decoder inputs to indicate the position of each word.



10

Transformers

Linear & Softmax Layer



Conversion to words

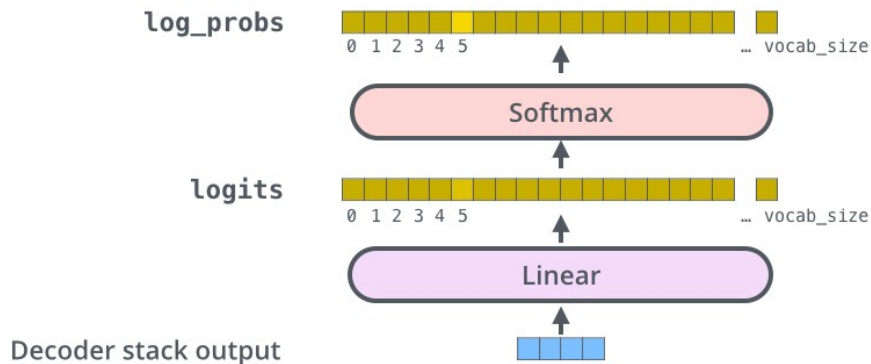
The decoder stack outputs a vector of floats. How do we turn that into a word? That's the job of the final Linear layer which is followed by a Softmax Layer.

The Linear layer is a simple fully connected neural network that projects the vector produced by the stack of decoders, into a much, much larger vector called a logits vector.

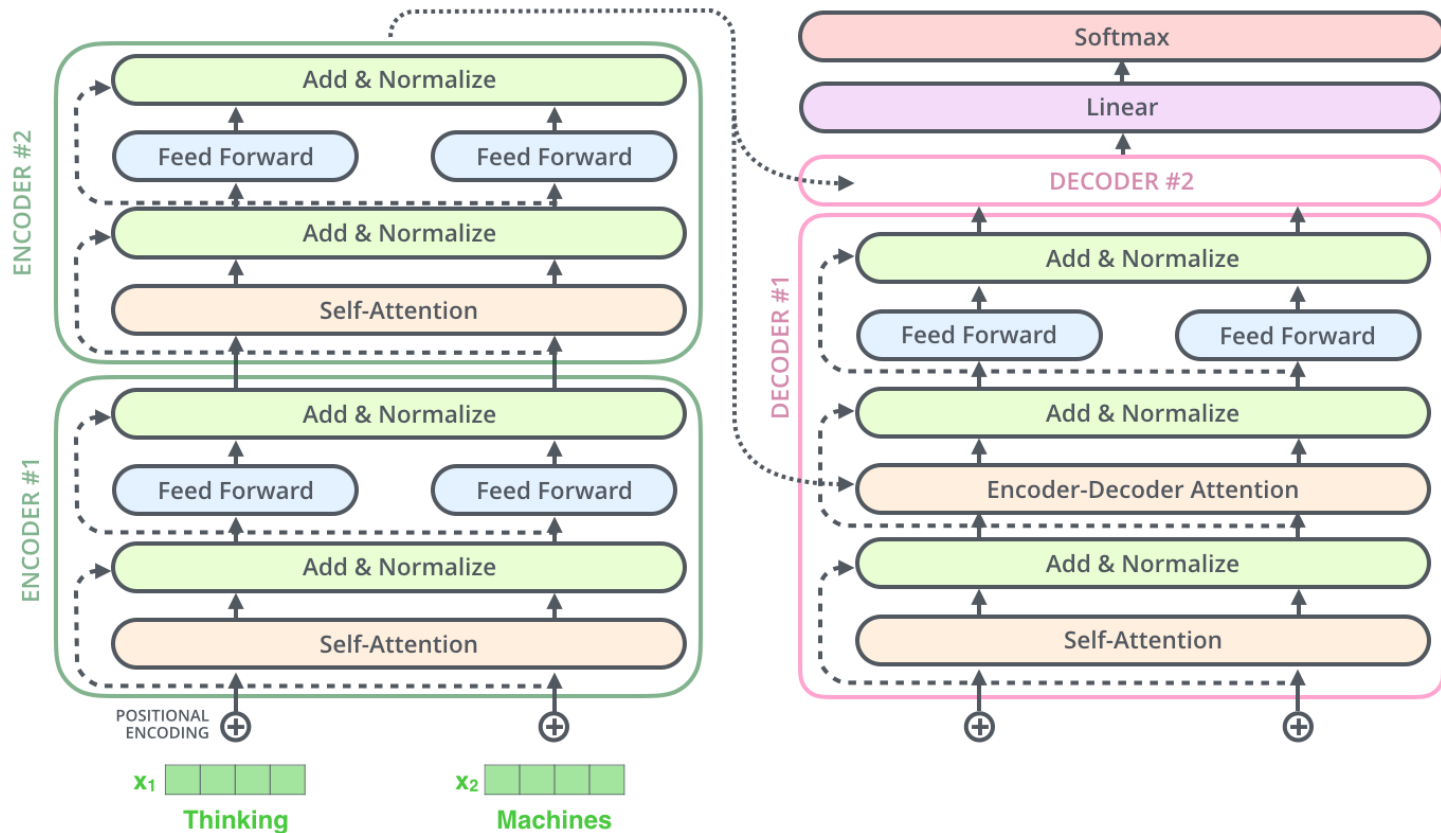
The softmax layer then turns those scores into probabilities (all positive, all add up to 1.0). The cell with the highest probability is chosen, and the word associated with it is produced as the output for this time step.

Which word in our vocabulary is associated with this index?

Get the index of the cell with the highest value (argmax)



Everything At Once!



THANKS!

Any questions?

You can find me at @sampurnrattan